

Sri Sivasubramaniya Nadar College of Engineering, Chennai

(An Autonomous Institution Affiliated to Anna University)

Machine Learning Algorithms Laboratory

Experiment 2

Email Spam/Ham Classification using Naïve Bayes and KNN

Name	MAHALINGAM S S
Register No.	3122247001034
Degree & Branch	M.Tech (Integrated) CSE
Semester	V
Subject Code	ICS1512
Academic Year	2026–2027
Batch	2024–2029

Aim

To classify emails as spam or ham using Naïve Bayes and K-Nearest Neighbors (KNN) algorithms and compare their performance.

Objective

- Implement Naïve Bayes and KNN classifiers.
- Compare Gaussian, Multinomial and Bernoulli Naïve Bayes.
- Study the effect of different values of k in KNN.
- Tune KNN using GridSearchCV and RandomizedSearchCV.
- Compare KDTree and BallTree.
- Evaluate computational complexity and execution time.
- Perform five-fold cross-validation.

Theory

Naïve Bayes

Naïve Bayes is a probabilistic supervised learning classifier based on Bayes' theorem. It assumes conditional independence between features given the class:

$$P(C | X) = \frac{P(X | C)P(C)}{P(X)}.$$

The experiment compares three variants: Gaussian, Multinomial and Bernoulli Naïve Bayes.

K-Nearest Neighbors

KNN is a lazy learning algorithm. For a new sample, the distances to training samples are calculated and the class is determined from the nearest k samples. The choice of k affects the bias–variance behaviour of the classifier.

KNN Search Structures

KDTree and BallTree are spatial data structures that can accelerate nearest-neighbor queries. Their theoretical complexity in the experiment is summarized later in the report.

Dataset

The Spambase dataset from Kaggle was used. It contains email samples represented by numerical features and a binary class label indicating spam or ham. The notebook records 4601 samples and 58 columns, including the class column.

Software Requirements

Python, NumPy, Pandas, Matplotlib and Scikit-learn were used in the notebook.

Implementation Steps

1. Load the dataset.

2. Inspect the dataset and handle/check missing values.
3. Normalize features using MinMaxScaler.
4. Perform exploratory data analysis.
5. Split the data into training and testing sets.
6. Train Gaussian, Multinomial and Bernoulli Naïve Bayes models.
7. Train KNN.
8. Evaluate the classifiers.
9. Study different k values.
10. Tune KNN using GridSearchCV and RandomizedSearchCV.
11. Compare KDTree and BallTree.
12. Perform five-fold cross-validation.
13. Measure training and prediction time.
14. Generate evaluation plots.

EDA

The notebook checks dataset shape, columns, data types, summary statistics, missing values and duplicate rows. It reports 391 duplicate rows. Features are normalized before model training.

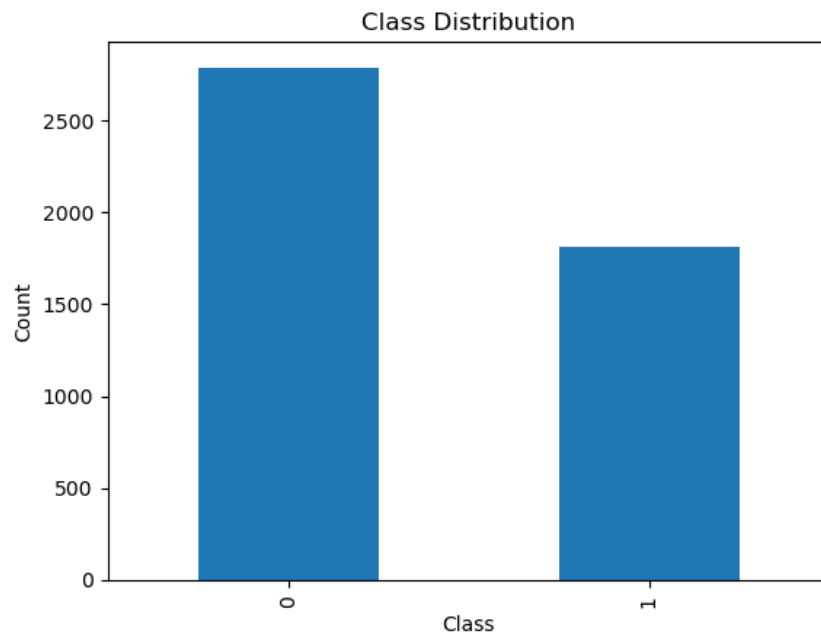


Figure 1: Class Distribution

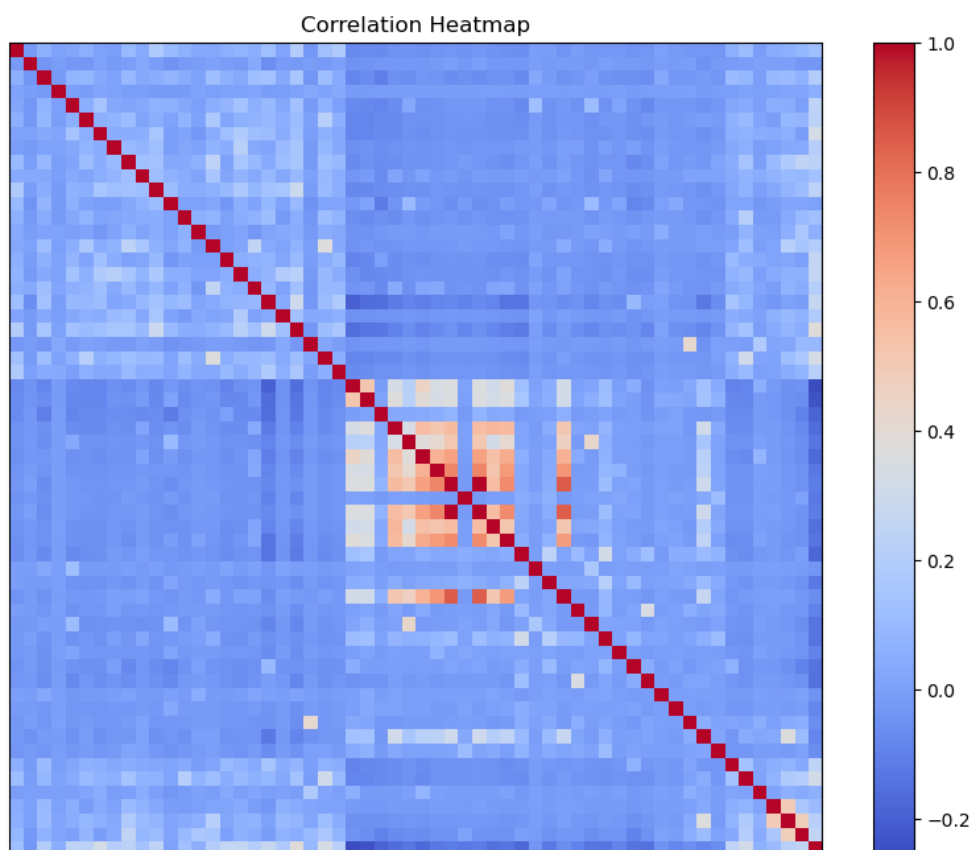


Figure 2: Correlation Heatmap

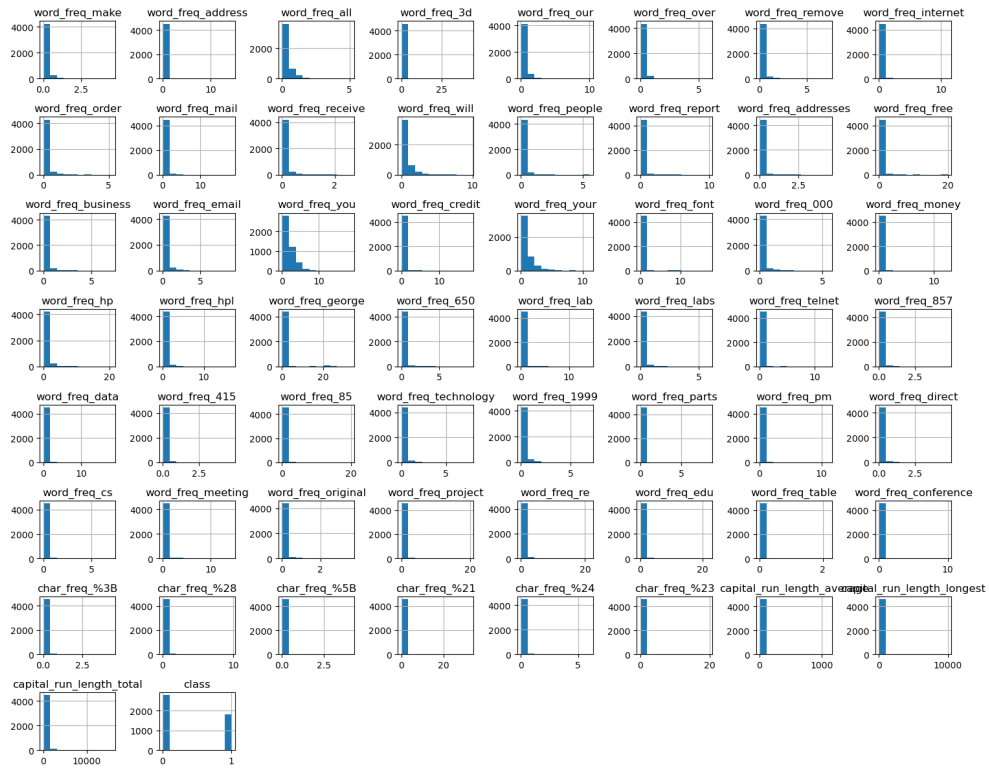


Figure 3: Feature Histograms

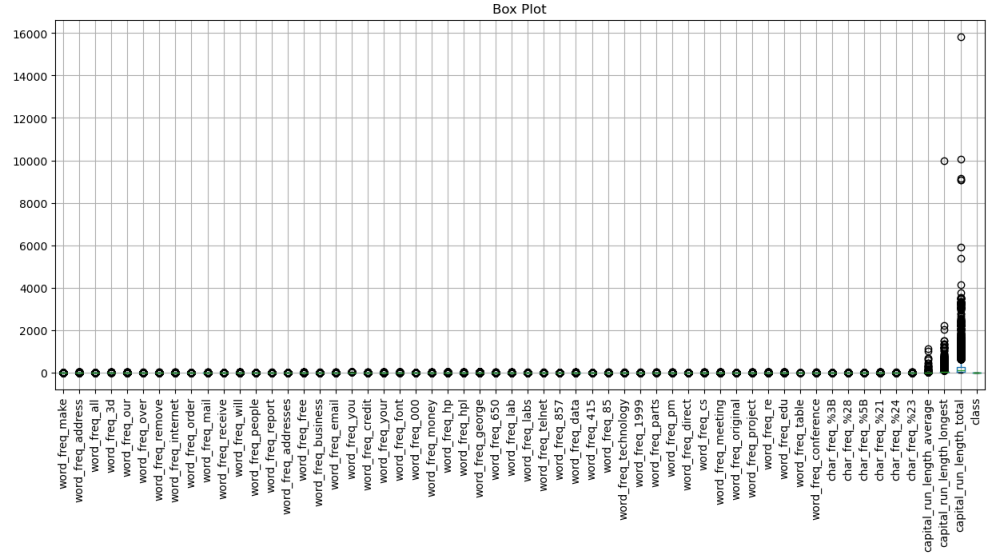


Figure 4: Feature Boxplots

Preprocessing and Data Split

The class column was separated from the input features. MinMaxScaler was used to normalize the features. The dataset was split using an 80:20 train-test split with random state 42. The

notebook reports 3680 training samples and 921 testing samples.

Naïve Bayes Comparison

The three Naïve Bayes classifiers were evaluated using accuracy, precision, recall, F1-score and ROC-AUC.

Table 1: Naïve Bayes Performance

Metric	Gaussian	Multinomial	Bernoulli
Accuracy	0.821933	0.871878	0.880565
Precision	0.723320	0.950331	0.904624
Recall	0.938462	0.735897	0.802564
F1	0.816964	0.829480	0.850543
ROC-AUC	0.837404	0.853824	0.870209

Among the three Naïve Bayes variants, Bernoulli Naïve Bayes achieved the highest accuracy, F1-score and ROC-AUC, while Multinomial Naïve Bayes achieved the highest precision. Gaussian Naïve Bayes achieved the highest recall.

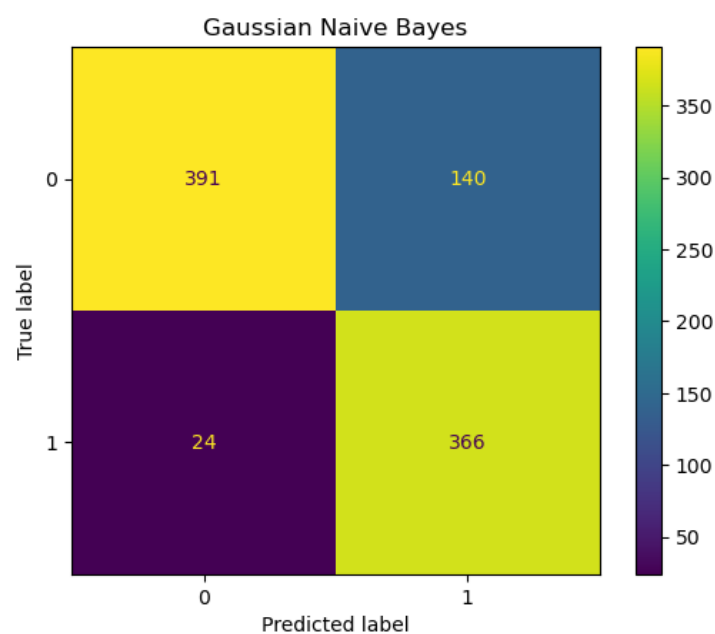


Figure 5: Gaussian Naïve Bayes Confusion Matrix

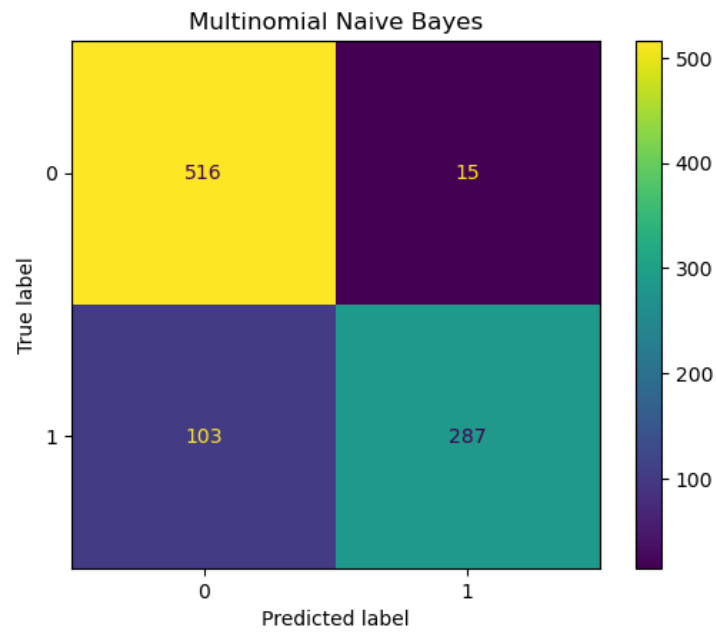


Figure 6: Multinomial Naïve Bayes Confusion Matrix

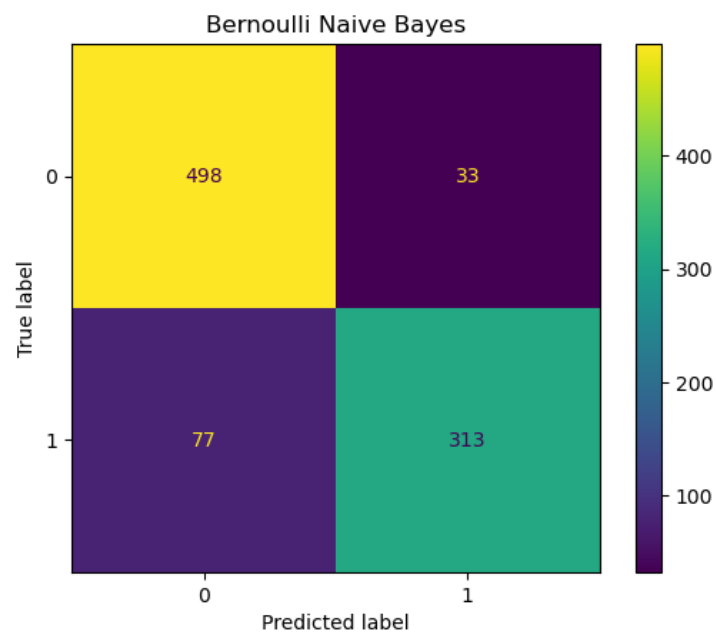


Figure 7: Bernoulli Naïve Bayes Confusion Matrix

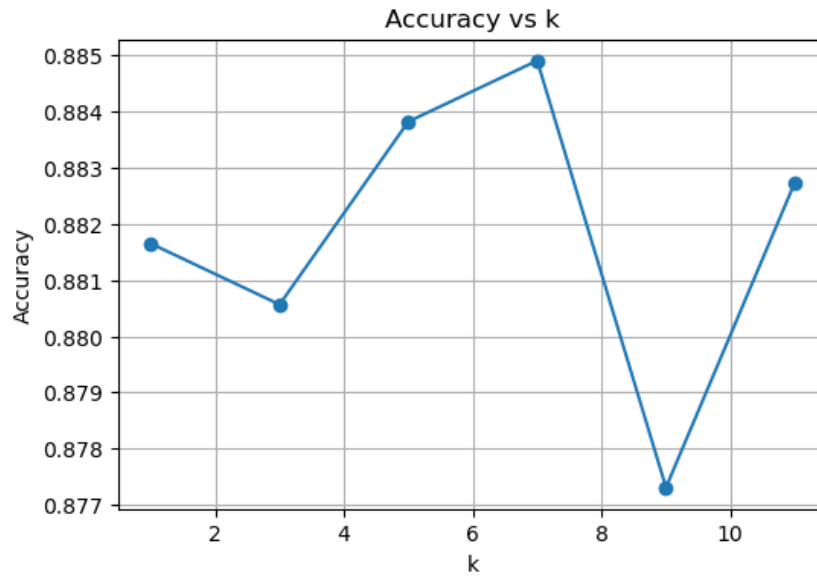
KNN Performance for Different k

The notebook evaluates $k = 1, 3, 5, 7, 9, 11$.

Table 2: KNN Comparison

k	Accuracy	Precision	Recall	F1
1	0.881650	0.850374	0.874359	0.862200
3	0.880565	0.872340	0.841026	0.856397
5	0.883822	0.877333	0.843590	0.860131
7	0.884908	0.879679	0.843590	0.861257
9	0.877307	0.887955	0.812821	0.848728
11	0.882736	0.896067	0.817949	0.855228

The highest test accuracy in the tested k values was obtained at $k = 7$ with 0.884908. However, GridSearchCV selected $k = 5$ with distance weighting based on five-fold cross-validation.

Figure 8: Accuracy vs k

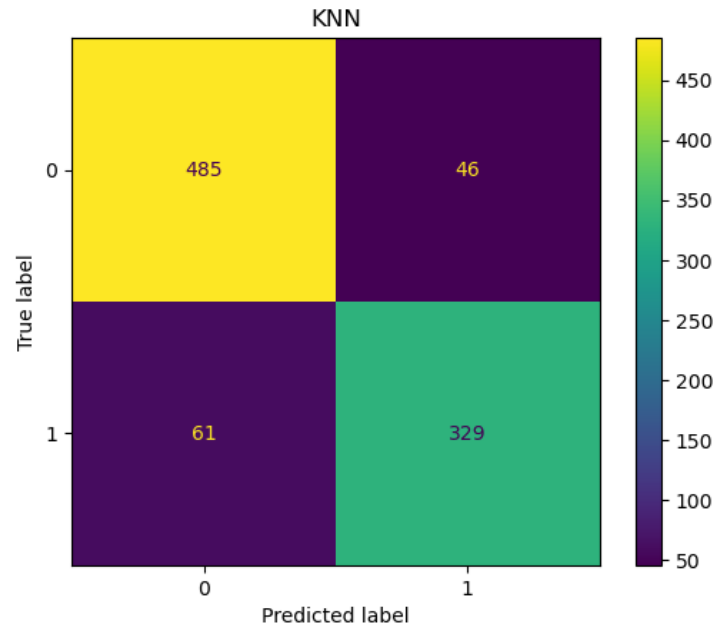


Figure 9: KNN Confusion Matrix

GridSearchCV vs RandomizedSearchCV

Both search methods were used with the same KNN parameter space:

- $n_neighbors \in \{1, 3, 5, 7, 9, 11\}$
- `weights` = uniform or distance
- `algorithm` = auto, kd_tree or ball_tree

Table 3: Grid Search vs Randomized Search

Parameter	GridSearchCV	RandomizedSearchCV
Best k	5	5
Weights	distance	distance
Algorithm	auto	kd_tree
CV Accuracy	0.914130	0.914130
Execution Time (s)	28.398	9.394

RandomizedSearchCV obtained the same cross-validation accuracy as GridSearchCV in the executed notebook while taking substantially less execution time because it evaluated only a limited number of sampled configurations.

KDTree vs BallTree

Both tree structures were evaluated with $k = 5$.

Table 4: KDTree vs BallTree

Metric	KDTree	BallTree
Accuracy	0.883822	0.883822
Training Time (s)	0.041	0.020
Prediction Time (s)	0.264	0.229

Both structures produced the same accuracy. In this executed run, BallTree had lower training and prediction time.

Five-Fold Cross Validation

The notebook compared Naïve Bayes with the best KNN configuration using five-fold cross-validation.

Table 5: Fold-wise Cross Validation Accuracy

Fold	Naïve Bayes	Best KNN
1	0.851249	0.876221
2	0.866304	0.892391
3	0.854348	0.918478
4	0.843478	0.909783
5	0.695652	0.763043
Average	0.822206	0.871983

Best KNN has the higher average cross-validation accuracy.

Theoretical Time Complexity

Table 6: Theoretical Complexity

Algorithm	Training	Prediction
Naïve Bayes	$O(nd)$	$O(d)$
KNN (Brute)	$O(1)$	$O(nd)$
KDTree	$O(n \log n)$	$O(\log n)$ average
BallTree	$O(n \log n)$	$O(\log n)$ average

Here n is the number of samples and d is the number of features. KNN has negligible model-building cost because it stores the training data, but prediction can be expensive because distances to training samples must be considered.

ROC and Precision–Recall Curves

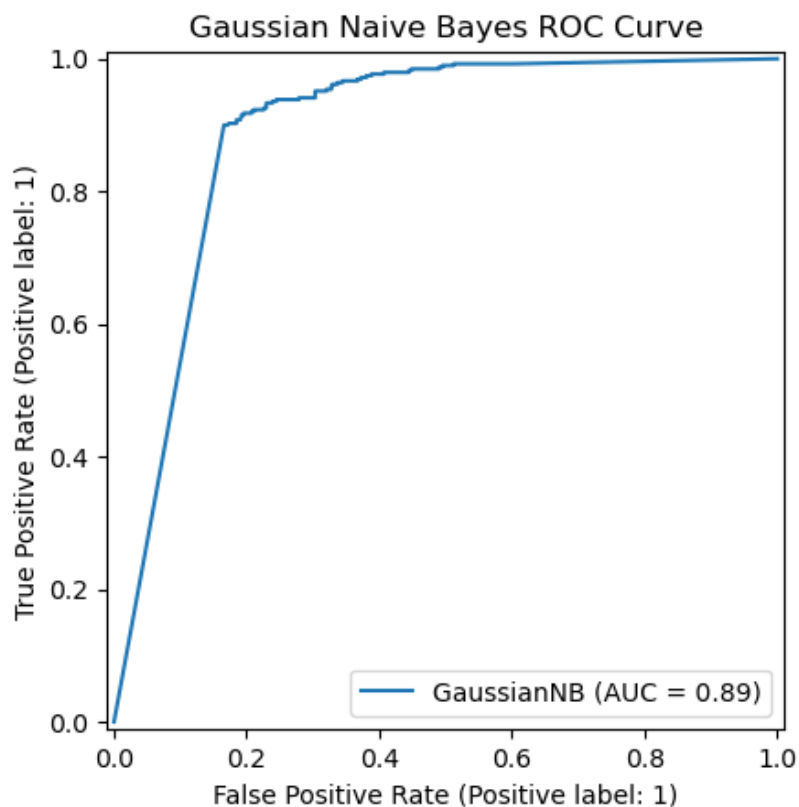


Figure 10: Gaussian Naïve Bayes ROC Curve

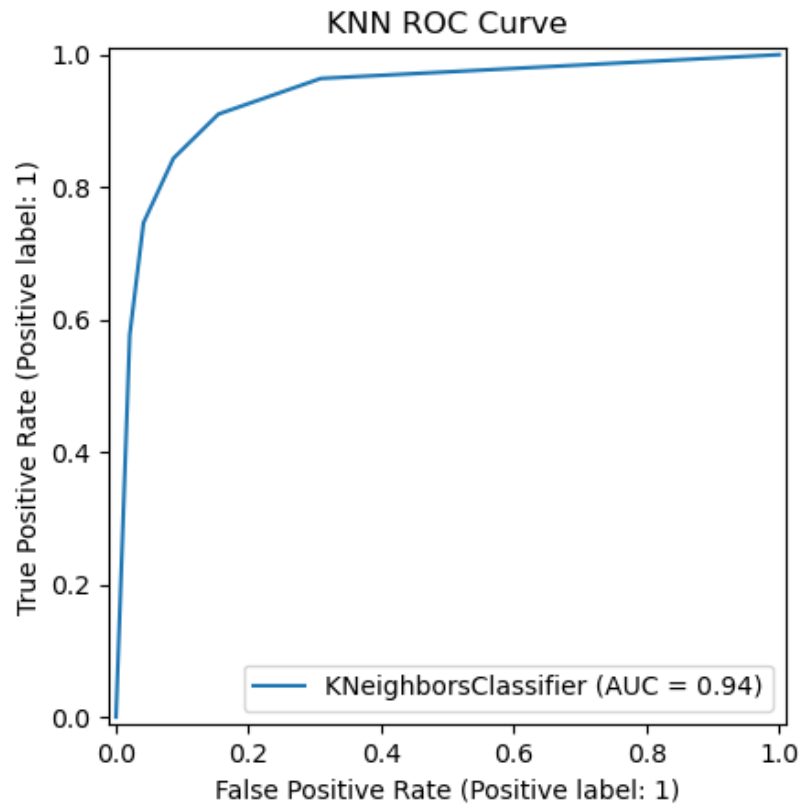


Figure 11: KNN ROC Curve

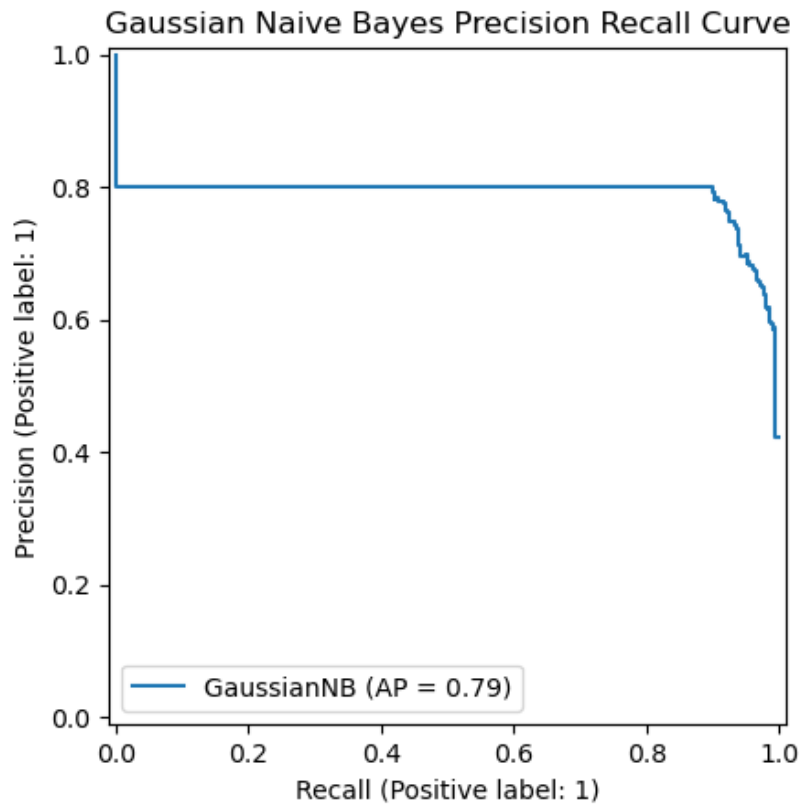


Figure 12: Gaussian Naïve Bayes Precision–Recall Curve

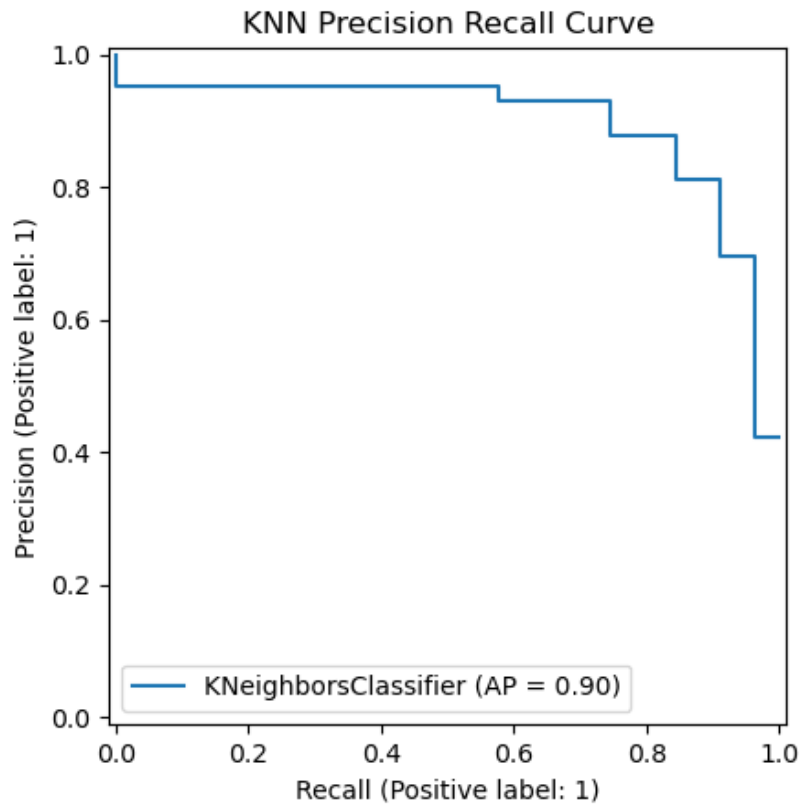


Figure 13: KNN Precision–Recall Curve

Training and Prediction Time

Table 7: Training Time

Algorithm	Training Time (s)
Gaussian NB	0.008
Multinomial NB	0.007
Bernoulli NB	0.012
Best KNN	0.002

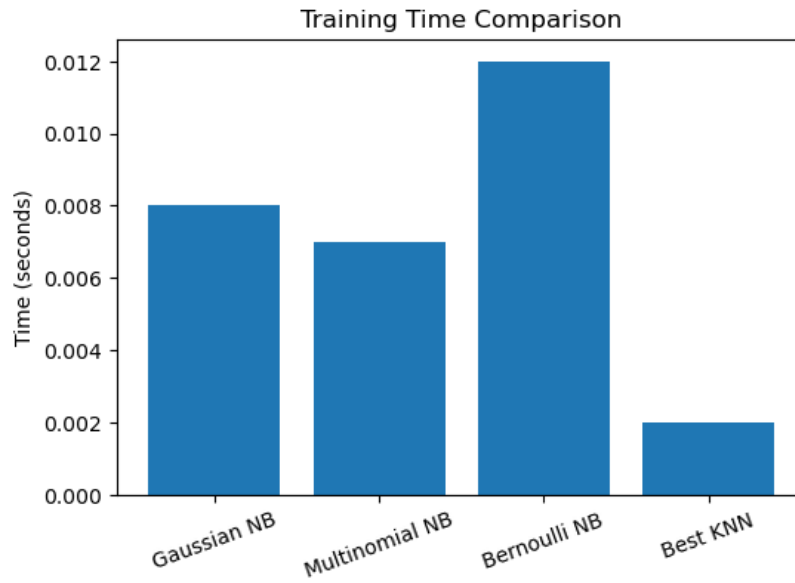


Figure 14: Training Time Comparison

Table 8: Prediction Time	
Algorithm	Prediction Time (s)
Gaussian NB	0.002
Multinomial NB	0.001
Bernoulli NB	0.002
Best KNN	0.111

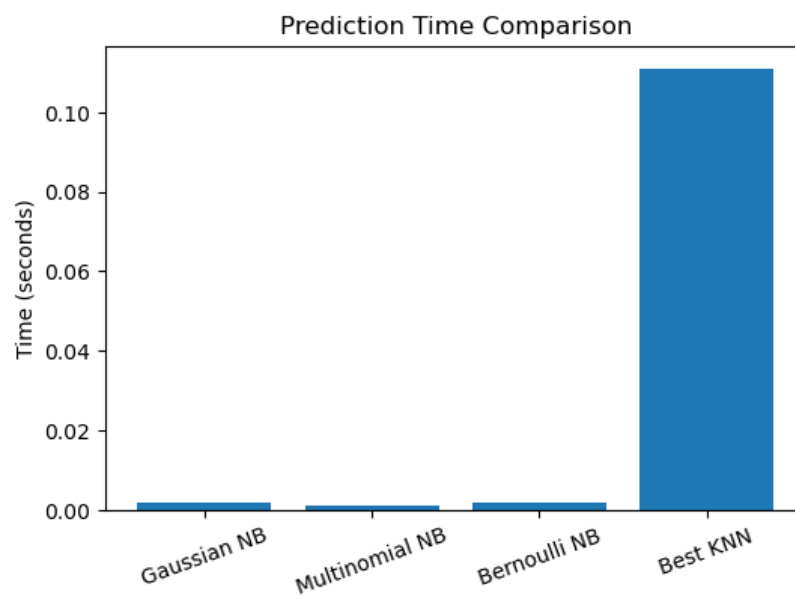


Figure 15: Prediction Time Comparison

Classifier Comparison

Table 9: Classifier Accuracy

Classifier	Accuracy
Gaussian NB	0.821933
Multinomial NB	0.871878
Bernoulli NB	0.880565
KNN	0.883822

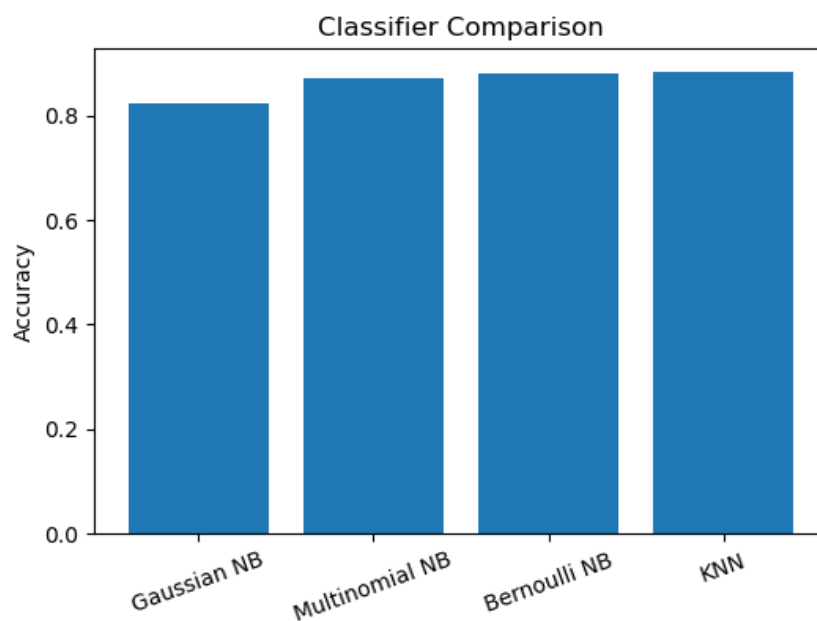


Figure 16: Classifier Accuracy Comparison

Observations and Analysis

1. Which Naïve Bayes variant performed best?

Bernoulli Naïve Bayes performed best among the three Naïve Bayes variants based on accuracy, F1-score and ROC-AUC. Its accuracy was 0.880565.

2. What is the optimal value of k ?

For the explicitly tested test-set values, $k = 7$ gave the highest accuracy of 0.884908. GridSearchCV, however, selected $k = 5$ with distance weighting based on cross-validation, so $k = 5$ is the selected tuned configuration.

3. Compare GridSearchCV and RandomizedSearchCV.

Both achieved a CV accuracy of 0.914130. GridSearchCV took 28.398 seconds, whereas RandomizedSearchCV took 9.394 seconds. Thus, RandomizedSearchCV was faster in this experiment.

4. Compare KDTree and BallTree.

Both produced an accuracy of 0.883822. BallTree had lower measured training time (0.020 s versus 0.041 s) and prediction time (0.229 s versus 0.264 s) in this run.

5. Compare theoretical and practical complexity.

Naïve Bayes has low prediction complexity because its model is directly parameterized. KNN has almost no training computation but a higher prediction cost. The measured results reflect this: Best KNN took only 0.002 s for training but 0.111 s for prediction.

6. Which classifier is preferred for large datasets?

For large datasets, a model with low prediction cost such as Naïve Bayes is generally attractive. KNN can become expensive at prediction time because it must search for nearest neighbors, although KDTree or BallTree can reduce the search cost for suitable data.

Additional Analysis

Train-test splits: The notebook uses an 80:20 split with random state 42. No additional split experiments were executed in the uploaded notebook, so no unsupported numerical comparison is included.

Euclidean and Manhattan distance: The uploaded notebook does not execute a separate Euclidean-versus-Manhattan comparison. Therefore, no numerical result is inserted for this task.

Weighted KNN: Weighted KNN is evaluated during GridSearchCV and RandomizedSearchCV through the `weights` parameter. The selected configuration uses `weights=distance`.

Overfitting and underfitting: Very small k values can make KNN sensitive to individual training samples and therefore have higher variance. Very large k values smooth the decision boundary and can increase bias. The tested accuracy values show that performance changes with k and peaks at $k = 7$ among the explicitly evaluated values.

Learning Outcomes

- Understood Bayes theorem and the Naïve Bayes classification assumption.
- Implemented Gaussian, Multinomial and Bernoulli Naïve Bayes.

- Implemented KNN and studied the effect of k .
- Used MinMaxScaler for feature normalization.
- Used GridSearchCV and RandomizedSearchCV for KNN tuning.
- Compared KDTree and BallTree.
- Performed five-fold cross-validation.
- Compared training and prediction time.
- Evaluated classifiers using accuracy, precision, recall, F1 and ROC-AUC.

Conclusion

Email Spam/Ham classification was performed using Naïve Bayes and KNN on the Spambase dataset. Gaussian, Multinomial and Bernoulli Naïve Bayes classifiers were compared, followed by KNN evaluation for different values of k . Bernoulli Naïve Bayes was the strongest Naïve Bayes variant, while KNN achieved the highest test accuracy among the classifiers shown in the notebook. GridSearchCV and RandomizedSearchCV selected $k = 5$ with distance weighting and achieved the same cross-validation accuracy, with RandomizedSearchCV requiring less execution time. Five-fold cross-validation also showed a higher average accuracy for the tuned KNN configuration than for Naïve Bayes.

Complete Python Code

The following section contains **all 42 code cells from the uploaded ML02.ipynb**, preserved in their original order.

```
% Code Cell 1
import time

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.model_selection import GridSearchCV
from sklearn.model_selection import RandomizedSearchCV
from sklearn.model_selection import cross_val_score

from sklearn.preprocessing import MinMaxScaler

from sklearn.naive_bayes import GaussianNB
from sklearn.naive_bayes import MultinomialNB
from sklearn.naive_bayes import BernoulliNB

from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score
from sklearn.metrics import roc_auc_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import ConfusionMatrixDisplay
from sklearn.metrics import RocCurveDisplay
from sklearn.metrics import PrecisionRecallDisplay

% Code Cell 2
df = pd.read_csv("spambase_csv.csv")

print("Dataset Loaded Successfully")

% Code Cell 3
df = pd.read_csv("spambase_csv.csv")

print("Dataset Loaded Successfully")

% Code Cell 4
df.head()
```

```

% Code Cell 5
print("Shape :", df.shape)

print("\nColumns")
print(df.columns)

print("\nData Types")
print(df.dtypes)

print("\nInformation")
df.info()

% Code Cell 6
df.describe()

% Code Cell 7
print(df.isnull().sum())

% Code Cell 8
print("Duplicate Rows :", df.duplicated().sum())

% Code Cell 9
df["class"].value_counts().plot(kind="bar")

plt.title("Class Distribution")
plt.xlabel("Class")
plt.ylabel("Count")

plt.show()

% Code Cell 10
plt.figure(figsize=(10,8))

corr = df.corr()

plt.imshow(corr, cmap="coolwarm")

plt.colorbar()

plt.xticks([])
plt.yticks([])

plt.title("Correlation Heatmap")

plt.show()

```

```

% Code Cell 11
df.hist(figsize=(15,12))

plt.tight_layout()

plt.show()

% Code Cell 12
plt.figure(figsize=(15,6))

df.boxplot(rot=90)

plt.title("Box Plot")

plt.show()

% Code Cell 13
X = df.drop("class", axis=1)
y = df["class"]

% Code Cell 14
scaler = MinMaxScaler()

X = scaler.fit_transform(X)

print("Features Normalized")

% Code Cell 15
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

print("Training Samples :", len(X_train))
print("Testing Samples :", len(X_test))

% Code Cell 16
gnb = GaussianNB()

start = time.time()

gnb.fit(X_train, y_train)

train_time_gnb = time.time() - start

```

```

start = time.time()

y_pred_gnb = gnb.predict(X_test)

prediction_time_gnb = time.time() - start

% Code Cell 17
mnb = MultinomialNB()

start = time.time()

mnb.fit(X_train, y_train)

train_time_mnb = time.time() - start

start = time.time()

y_pred_mnb = mnb.predict(X_test)

prediction_time_mnb = time.time() - start

% Code Cell 18
bnb = BernoulliNB()

start = time.time()

bnb.fit(X_train, y_train)

train_time_bnb = time.time() - start

start = time.time()

y_pred_bnb = bnb.predict(X_test)

prediction_time_bnb = time.time() - start

% Code Cell 19
knn = KNeighborsClassifier(n_neighbors=5)

start = time.time()

knn.fit(X_train, y_train)

train_time_knn = time.time() - start

start = time.time()

```

```

y_pred_knn = knn.predict(X_test)

prediction_time_knn = time.time() - start

% Code Cell 20
def evaluate(y_true, y_pred):

    accuracy = accuracy_score(y_true, y_pred)
    precision = precision_score(y_true, y_pred)
    recall = recall_score(y_true, y_pred)
    f1 = f1_score(y_true, y_pred)
    roc = roc_auc_score(y_true, y_pred)

    return accuracy, precision, recall, f1, roc

% Code Cell 21
g = evaluate(y_test, y_pred_gnb)
m = evaluate(y_test, y_pred_mnb)
b = evaluate(y_test, y_pred_bnb)

nb_results = pd.DataFrame({
    "Metric": ["Accuracy", "Precision", "Recall", "F1", "ROC-AUC"],
    "Gaussian": g,
    "Multinomial": m,
    "Bernoulli": b
})

nb_results

% Code Cell 22
ConfusionMatrixDisplay.from_predictions(y_test, y_pred_gnb)

plt.title("Gaussian Naive Bayes")

plt.show()

% Code Cell 23
ConfusionMatrixDisplay.from_predictions(y_test, y_pred_mnb)

plt.title("Multinomial Naive Bayes")

plt.show()

% Code Cell 24
ConfusionMatrixDisplay.from_predictions(y_test, y_pred_bnb)

plt.title("Bernoulli Naive Bayes")

```

```

plt.show()

% Code Cell 25
ConfusionMatrixDisplay.from_predictions(y_test, y_pred_knn)

plt.title("KNN")

plt.show()

% Code Cell 26
k_values = [1, 3, 5, 7, 9, 11]

results = []

for k in k_values:

    model = KNeighborsClassifier(n_neighbors=k)

    model.fit(X_train, y_train)

    pred = model.predict(X_test)

    accuracy = accuracy_score(y_test, pred)
    precision = precision_score(y_test, pred)
    recall = recall_score(y_test, pred)
    f1 = f1_score(y_test, pred)

    results.append([k, accuracy, precision, recall, f1])

knn_table = pd.DataFrame(
    results,
    columns=["k", "Accuracy", "Precision", "Recall", "F1"]
)

knn_table

% Code Cell 27
plt.figure(figsize=(6,4))

plt.plot(
    knn_table["k"],
    knn_table["Accuracy"],
    marker="o"
)

plt.title("Accuracy vs k")

```

```

plt.xlabel("k")

plt.ylabel("Accuracy")

plt.grid(True)

plt.show()

% Code Cell 28
parameters = {
    "n_neighbors": [1, 3, 5, 7, 9, 11],
    "weights": ["uniform", "distance"],
    "algorithm": ["auto", "kd_tree", "ball_tree"]
}

start = time.time()

grid_search = GridSearchCV(
    KNeighborsClassifier(),
    parameters,
    cv=5
)

grid_search.fit(X_train, y_train)

grid_time = time.time() - start

print("Best Parameters")
print(grid_search.best_params_)

print("\nBest CV Score")
print(grid_search.best_score_)

% Code Cell 29
random_search = RandomizedSearchCV(
    KNeighborsClassifier(),
    param_distributions=parameters,
    n_iter=10,
    cv=5,
    random_state=42
)

start = time.time()

random_search.fit(X_train, y_train)

```

```

random_time = time.time() - start

print("Best Parameters")
print(random_search.best_params_)

print("\nBest CV Score")
print(random_search.best_score_)

% Code Cell 30
search_result = pd.DataFrame({
    "Parameter": [
        "Best k",
        "Weights",
        "Algorithm",
        "CV Accuracy",
        "Execution Time"
    ],
    "GridSearchCV": [
        grid_search.best_params_["n_neighbors"],
        grid_search.best_params_["weights"],
        grid_search.best_params_["algorithm"],
        grid_search.best_score_,
        grid_time
    ],
    "RandomizedSearchCV": [
        random_search.best_params_["n_neighbors"],
        random_search.best_params_["weights"],
        random_search.best_params_["algorithm"],
        random_search.best_score_,
        random_time
    ]
})

search_result

% Code Cell 31
algorithms = ["kd_tree", "ball_tree"]

tree_result = []

for algo in algorithms:

    model = KNeighborsClassifier(
        n_neighbors=5,
        algorithm=algo
    )

```

```

start = time.time()
model.fit(X_train, y_train)
train_time = time.time() - start

start = time.time()
pred = model.predict(X_test)
predict_time = time.time() - start

accuracy = accuracy_score(y_test, pred)

tree_result.append(
    [algo, accuracy, train_time, predict_time]
)

tree_table = pd.DataFrame(
    tree_result,
    columns=[
        "Algorithm",
        "Accuracy",
        "Training Time",
        "Prediction Time"
    ]
)

tree_table

% Code Cell 32
nb_cv = cross_val_score(GaussianNB(), X, y, cv=5)

best_knn = KNeighborsClassifier(
    n_neighbors=grid_search.best_params_["n_neighbors"],
    weights=grid_search.best_params_["weights"],
    algorithm=grid_search.best_params_["algorithm"]
)

knn_cv = cross_val_score(best_knn, X, y, cv=5)

cv_table = pd.DataFrame({
    "Fold": [1, 2, 3, 4, 5],
    "Naive Bayes": nb_cv,
    "Best KNN": knn_cv
})

cv_table

% Code Cell 33
average = pd.DataFrame({

```

```

        "Model":["Naive Bayes","Best KNN"],
        "Average Accuracy":[
            nb_cv.mean(),
            knn_cv.mean()
        ]
    })

average

% Code Cell 34
RocCurveDisplay.from_estimator(gnb, X_test, y_test)
plt.title("Gaussian Naive Bayes ROC Curve")
plt.show()

RocCurveDisplay.from_estimator(knn, X_test, y_test)
plt.title("KNN ROC Curve")
plt.show()

% Code Cell 35
PrecisionRecallDisplay.from_estimator(gnb, X_test, y_test)
plt.title("Gaussian Naive Bayes Precision Recall Curve")
plt.show()

PrecisionRecallDisplay.from_estimator(knn, X_test, y_test)
plt.title("KNN Precision Recall Curve")
plt.show()

% Code Cell 36
training = pd.DataFrame({
    "Algorithm":[
        "Gaussian NB",
        "Multinomial NB",
        "Bernoulli NB",
        "Best KNN"
    ],
    "Training Time":[
        train_time_gnb,
        train_time_mnb,
        train_time_bnb,
        train_time_knn
    ]
})

training

% Code Cell 37
prediction = pd.DataFrame({

```

```

        "Algorithm":[
            "Gaussian NB",
            "Multinomial NB",
            "Bernoulli NB",
            "Best KNN"
        ],
        "Prediction Time":[
            prediction_time_gnb,
            prediction_time_mnb,
            prediction_time_bnb,
            prediction_time_knn
        ]
    })

prediction

% Code Cell 38
plt.figure(figsize=(6,4))

plt.bar(training["Algorithm"], training["Training Time"])

plt.title("Training Time Comparison")

plt.ylabel("Time (seconds)")

plt.xticks(rotation=20)

plt.show()

% Code Cell 39
plt.figure(figsize=(6,4))

plt.bar(prediction["Algorithm"], prediction["Prediction Time"])

plt.title("Prediction Time Comparison")

plt.ylabel("Time (seconds)")

plt.xticks(rotation=20)

plt.show()

% Code Cell 40
classifier = pd.DataFrame({
    "Classifier":[
        "Gaussian NB",
        "Multinomial NB",

```

```

        "Bernoulli NB",
        "KNN"
    ],
    "Accuracy": [
        g[0],
        m[0],
        b[0],
        evaluate(y_test, y_pred_knn)[0]
    ]
}))

classifier

% Code Cell 41
plt.figure(figsize=(6,4))

plt.bar(classifier["Classifier"], classifier["Accuracy"])

plt.title("Classifier Comparison")

plt.ylabel("Accuracy")

plt.xticks(rotation=20)

plt.show()

% Code Cell 42

```

References

- Scikit-learn Documentation.
- Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow.
- Pattern Recognition and Machine Learning.
- Spambase Dataset from Kaggle.